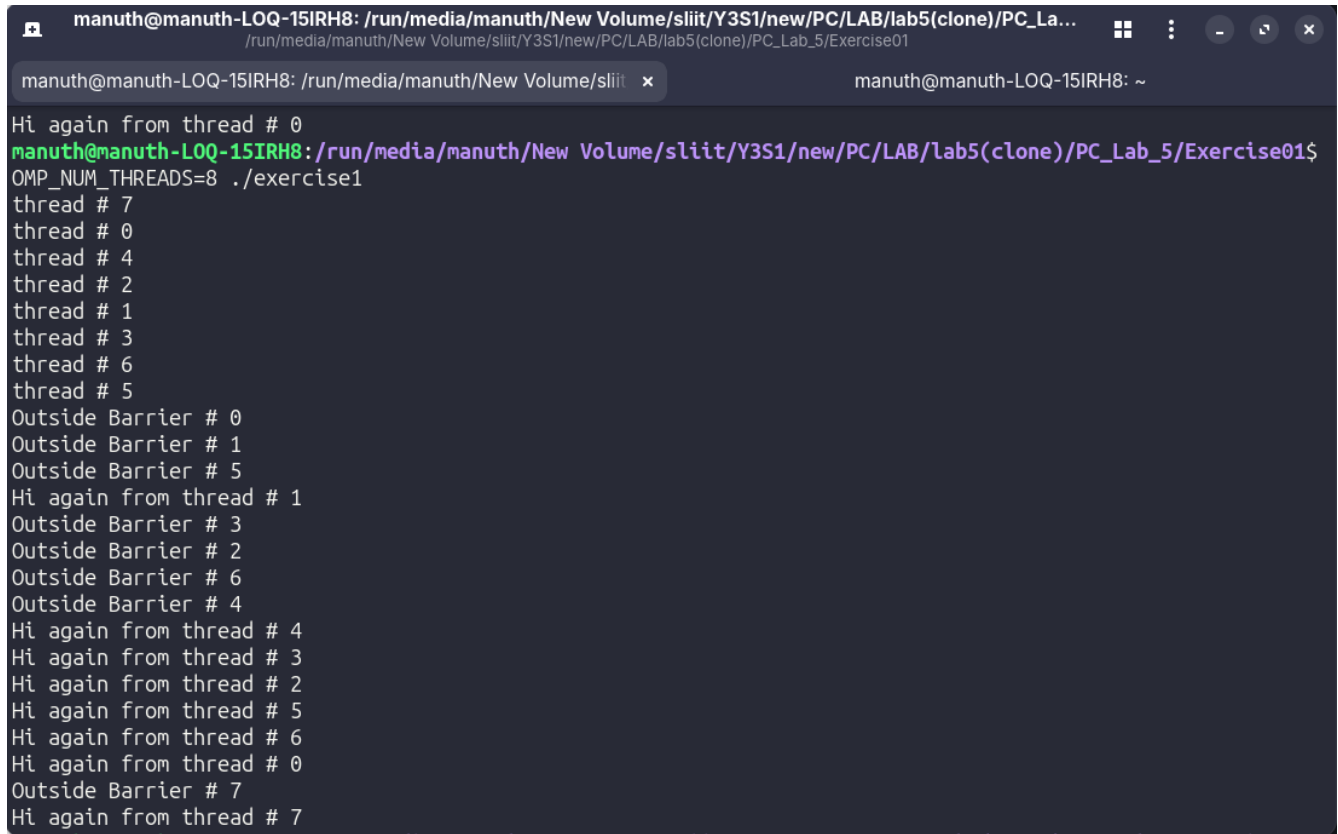


PC LAB 6

Name – Wilegoda M.H
IT Number = IT24101472

Exercise 1



```
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_La...
/run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise01
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volume/sliit x manuth@manuth-LOQ-15IRH8: ~
Hi again from thread # 0
manuth@manuth-LOQ-15IRH8:/run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise01$
OMP_NUM_THREADS=8 ./exercise1
thread # 7
thread # 0
thread # 4
thread # 2
thread # 1
thread # 3
thread # 6
thread # 5
Outside Barrier # 0
Outside Barrier # 1
Outside Barrier # 5
Hi again from thread # 1
Outside Barrier # 3
Outside Barrier # 2
Outside Barrier # 6
Outside Barrier # 4
Hi again from thread # 4
Hi again from thread # 3
Hi again from thread # 2
Hi again from thread # 5
Hi again from thread # 6
Hi again from thread # 0
Outside Barrier # 7
Hi again from thread # 7
```

#pragma omp barrier synchronizes all threads. Each thread waits at the barrier until every thread in the parallel region reaches it. After all threads arrive, they are released and continue execution. The order after the barrier is still not guaranteed.

Exercise 2

```
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise02$ OMP_NUM_THREADS=4 ./exercise2
thread # 3
Inside Single # 3
Exiting Single
thread # 2
thread # 1
thread # 0
Hi again from thread # 0
Hi again from thread # 1
Hi again from thread # 3
Hi again from thread # 2
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise02$
```

#pragma omp single allows only one thread in the parallel region to execute the enclosed block, while the other threads skip it. In this program, whichever thread reaches the single section first executes the Inside Single and Exiting Single statements. After that block, there is an implicit barrier by default, so the other threads wait until the single section is completed before all threads continue to print the final Hi again message.

Exercise 3

```
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise03$ gcc -fopenmp exercise3.c -o exercise3
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise03$ OMP_NUM_THREADS=4 ./exercise3
thread # 0
Inside Master # 0
Exiting Master
Hi again from thread # 0
thread # 3
Hi again from thread # 3
thread # 2
Hi again from thread # 2
thread # 1
Hi again from thread # 1
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise03$ OMP_NUM_THREADS=4 ./exercise3
thread # 3
thread # 0
Inside Master # 0
Exiting Master
Hi again from thread # 0
thread # 1
Hi again from thread # 1
Hi again from thread # 3
thread # 2
Hi again from thread # 2
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise03$
```

#pragma omp master allows only the master thread, which is always **Thread 0**, to execute the enclosed block. All other threads skip that block and continue with the following statements. Unlike #pragma omp single, the master directive does **not** have an implicit barrier, so the other threads do not wait for Thread 0 to finish the master section before continuing.

Exercise 4

using #pragma omp for

```
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/...  
/run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise04  
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Vc x manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volun  
manuth@manuth-LOQ-15IRH8:/run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise04$ OMP_NUM_THREADS=4 ./exercise4  
r = 5, k = 1, thread # = 3  
r = 5, k = 2, thread # = 3  
r = 5, k = 3, thread # = 3  
r = 5, k = 4, thread # = 3  
r = 5, k = 5, thread # = 3  
r = 5, k = 6, thread # = 3  
r = 5, k = 7, thread # = 3  
r = 5, k = 8, thread # = 3  
r = 5, k = 9, thread # = 3  
r = 5, k = 10, thread # = 3  
r = 4, k = 1, thread # = 2  
r = 4, k = 2, thread # = 2  
r = 4, k = 3, thread # = 2  
r = 4, k = 4, thread # = 2  
r = 4, k = 5, thread # = 2  
r = 4, k = 6, thread # = 2  
r = 4, k = 7, thread # = 2  
r = 4, k = 8, thread # = 2  
r = 4, k = 9, thread # = 2  
r = 4, k = 10, thread # = 2  
r = 1, k = 1, thread # = 0  
r = 1, k = 2, thread # = 0  
r = 1, k = 3, thread # = 0  
r = 1, k = 4, thread # = 0  
r = 1, k = 5, thread # = 0  
r = 1, k = 6, thread # = 0  
r = 3, k = 1, thread # = 1  
r = 3, k = 2, thread # = 1  
r = 1, k = 7, thread # = 0  
r = 1, k = 8, thread # = 0  
r = 1, k = 9, thread # = 0  
r = 1, k = 10, thread # = 0  
r = 2, k = 1, thread # = 0  
r = 2, k = 2, thread # = 0  
r = 2, k = 3, thread # = 0  
r = 2, k = 4, thread # = 0  
r = 3, k = 3, thread # = 1
```

#pragma omp parallel for
→ parallelizes outer loop only
→ 5 outer iterations distributed

using #pragma omp parallel for collapse(2)

```
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/...  
/run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise04  
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Vc x manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volun  
manuth@manuth-LOQ-15IRH8:/run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise04$ gcc -fopenmp exercise4.c -o exercise4  
manuth@manuth-LOQ-15IRH8:/run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise04$ OMP_NUM_THREADS=4 ./exercise4  
r = 2, k = 4, thread # = 1  
r = 2, k = 5, thread # = 1  
r = 2, k = 6, thread # = 1  
r = 2, k = 7, thread # = 1  
r = 2, k = 8, thread # = 1  
r = 2, k = 9, thread # = 1  
r = 2, k = 10, thread # = 1  
r = 3, k = 1, thread # = 1  
r = 3, k = 2, thread # = 1  
r = 3, k = 3, thread # = 1  
r = 3, k = 4, thread # = 1  
r = 3, k = 5, thread # = 1  
r = 3, k = 6, thread # = 1  
r = 3, k = 7, thread # = 2  
r = 3, k = 8, thread # = 2  
r = 3, k = 9, thread # = 2  
r = 4, k = 9, thread # = 3  
r = 4, k = 10, thread # = 3  
r = 5, k = 1, thread # = 3  
r = 5, k = 2, thread # = 3  
r = 5, k = 3, thread # = 3  
r = 5, k = 4, thread # = 3  
r = 5, k = 5, thread # = 3  
r = 5, k = 6, thread # = 3  
r = 3, k = 10, thread # = 2  
r = 1, k = 1, thread # = 0  
r = 1, k = 2, thread # = 0  
r = 1, k = 3, thread # = 0  
r = 1, k = 4, thread # = 0  
r = 1, k = 5, thread # = 0  
r = 1, k = 6, thread # = 0  
r = 1, k = 7, thread # = 0  
r = 1, k = 8, thread # = 0  
r = 4, k = 1, thread # = 2  
r = 4, k = 2, thread # = 2
```

The key thing to observe is that without collapse(2), one thread normally handles a whole group of k values for one r; with collapse(2), the 50 (r,k) iterations can be distributed more evenly across the threads.

Exercise 5

```
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise05$ gcc -fopenmp exercise5.c -o exercise5
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise05$ OMP_NUM_THREADS=4 ./exercise5
Fibonacci(10) = 55
```

Exercise 6

```
5/Exercise06$ gcc -fopenmp exercise6.c -o exercise6
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise06$ OMP_NUM_THREADS=4 ./exercise6
C[0] = 6.00
C[999999] = 6.00
Execution Time = 0.001919 seconds
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise06$
```

Strip mining transforms a large loop into an outer loop over fixed-size blocks and an inner loop over the elements inside each block. OpenMP can distribute the strips among threads, while SIMD can efficiently process the elements within each strip.

Exercise 7

```
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise07$ ./mandelbrot_serial
Area of Mandelbrot set = 1.50928453 +/- 0.00075464
Execution time = 5.843676 seconds
manuth@manuth-LOQ-15IRH8: /run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_5/Exercise07$
```

```

manuth@manuth-L0Q-15IRH8:/run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_
5/Exercise07$ gcc -O2 -fopenmp mandelbrot_parallel.c -o mandelbrot_parallel -lm
manuth@manuth-L0Q-15IRH8:/run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_
5/Exercise07$ OMP_NUM_THREADS=1 ./mandelbrot_parallel
Area of Mandelbrot set = 1.50928453 +/- 0.00075464
Execution time = 6.311805 seconds
manuth@manuth-L0Q-15IRH8:/run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_
5/Exercise07$ OMP_NUM_THREADS=2 ./mandelbrot_parallel
Area of Mandelbrot set = 1.50928453 +/- 0.00075464
Execution time = 2.968463 seconds
manuth@manuth-L0Q-15IRH8:/run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_
5/Exercise07$ OMP_NUM_THREADS=3 ./mandelbrot_parallel
Area of Mandelbrot set = 1.50928453 +/- 0.00075464
Execution time = 2.022502 seconds
manuth@manuth-L0Q-15IRH8:/run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_
5/Exercise07$ OMP_NUM_THREADS=4 ./mandelbrot_parallel
Area of Mandelbrot set = 1.50928453 +/- 0.00075464
Execution time = 1.493842 seconds
manuth@manuth-L0Q-15IRH8:/run/media/manuth/New Volume/sliit/Y3S1/new/PC/LAB/lab5(clone)/PC_Lab_
5/Exercise07$

```

Version	Threads	Area	Time
Serial	1	1.50928453	5.843676 s
Parallel	1	1.50928453	6.311805 s
Parallel	2	1.50928453	2.968463 s
Parallel	3	1.50928453	2.022502 s
Parallel	4	1.50928453	1.493842 s

The Mandelbrot program was parallelized using an OpenMP parallel region and the outer loop was distributed among multiple threads. The numoutside variable was handled using a reduction to avoid race conditions. Testing with 1, 2, 3, and 4 threads produced the same Mandelbrot area of 1.50928453, confirming that the parallelization was correct. The execution time decreased from approximately 6.31 seconds with one thread to 1.49 seconds with four threads, showing a significant improvement in performance as the number of threads increased.